

Retro Tron Battle

Server Documentation

A network lightcycle duel for real retro computers (Atari, Commodore 64, Schneider/Amstrad CPC) with server-side rendering.

Built for [Classic Computing 2026](#) in Celle (10-11 October 2026, Hall 10 / CD-Kaserne).

This document describes the Python server (`tron_server.py`), the two participation methods for retro clients, the hardware interfaces used, how to test without real hardware, and how to extend the setup to further platforms.

1. Overview

Retro Tron Battle is a multiplayer lightcycle game (in the style of "Tron") in which real retro computers compete against each other via their network add-ons. The twist: the retro computers don't draw the playfield themselves - they only send the steering direction. The complete playfield is drawn centrally on the PC running the server (via a pygame window, well suited for a projector/large monitor at an event).

Currently supported platforms:

- **Atari XL/XE** with FujiNet network card - connects via a real TCP socket (port 6502)
- **Commodore 64** with Meatloaf interface (standard client) - connects via HTTP polling (port 8080)
- **Commodore 64** with WiC64 interface (experimental client, User Port instead of the IEC bus) - also via HTTP polling (port 8080)
- **Schneider/Amstrad CPC** with M4 board - also connects via HTTP polling (port 8080)

Why different methods per platform? That's not a design choice, it's a technical necessity - see section 3.

1.1 Architecture in brief

- The server (**tron_server.py**) runs the entire game logic: movement, collision detection, score.
- A **pygame window** shows the playfield, visitor photos, company logos, and a demo scroller.
- Each retro client does almost nothing except: read the joystick, send direction changes, occasionally poll whether the game is over.
- Results are continuously written to a CSV file and read back in at server startup (the score survives restarts).
- All paths (font, logos, results CSV, photo cache) are resolved relative to tron_server.py's location, not the current working directory - so the repo works unchanged no matter where the server is started from.
- Convention: each client reads the **first joystick port** of its computer (Atari STICK(0), CPC JOY(0), C64 PEEK(56321)/\$DC01).

2. Game flow

- A player connects (TCP or HTTP, see section 3) and optionally sends a visitor PIN for their event photo.
- The server waits until a second player joins.
- Once two players are present: a short countdown in the window (3, 2, 1, GO!) - direction changes are already accepted, but nobody moves yet.
- After GO!, actual movement starts, at an adjustable speed (TICK_RATE).
- The game ends on a collision (wall, own trail, opponent's trail) - a win, or a draw on a simultaneous crash.
- The result is sent to both clients, written to the CSV, and the server returns to the waiting state (the demo scroller starts up again).

3. Two participation methods

Not every retro network solution can keep a persistent TCP connection open from BASIC. FujiNet (Atari) can do this reliably - Meatloaf and WiC64 (both C64) as well as the M4 board (Amstrad CPC) are instead built for simple, one-off HTTP requests. A raw socket is either technically impossible from BASIC there (M4, without Z80 assembly) or has proven unreliable in practice over many rounds of testing (Meatloaf). Both paths run through exactly the same game logic (run_game()) server-side - it's just a different "front door".

3.1 Method 1: raw TCP (port 6502)

Used e.g. by the Atari with FujiNet. One connection stays open for the whole game.

Client → Server

Command	Description
HELLO <PLATFORM> <NAME> [PIN]	Login, PIN optional (event photo ID)
MOVE <U D L R>	Direction change (a direct reversal into your own trail is ignored)
BYE	End connection/game

Server → Client

Message	Description
WAIT	Connected, waiting for an opponent
START <w> <h> <x1> <y1> <x2> <y2> <yourplayernum> <p1plat> <p2plat>	Steering may begin now
END WIN <PLATFORM> <NAME>	Game over, someone won
END DRAW	Draw (simultaneous crash)
STATS GAMES <n> ... DRAWS <n>	Running totals, sent right after END

There is no TICK broadcast in this protocol - the playfield only exists in the server process and on screen. The client therefore almost never needs to read, which sidesteps the problem that FujiNet connections can become unreliable when reads/writes on the same channel get nested too tightly.

3.2 Method 2: HTTP polling (port 8080)

Used by the C64 (Meatloaf and WiC64) and the Schneider CPC (M4 board). Instead of a persistent connection, the client repeatedly polls two simple HTTP GET routes:

Route	Response
GET /join/<platform>/<name>/<pin-or-NONE>	"SESSION <id>" + WAIT, or straight to START if paired immediately

GET /tick/<session>/<direction-or-N>	Whatever is pending since the last poll (empty, START, END+STATS), or "ERR UNKNOWN SESSION" if the session is gone
--------------------------------------	--

A session behaves like a mailbox: /join creates one, every /tick call both delivers a direction change and picks up whatever the server has queued since then. After the game ends, the session stays valid for **15 seconds** (grace period), so even a slow, real HTTP round trip can still pick up the final result before the server cleans up.

Important lesson from practice: letter case

Testing showed that Meatloaf actually converts received HTTP content to lowercase when reading it (not just a display quirk, a real byte conversion). The server therefore deliberately compares session IDs case-insensitively, and the clients check keywords (START/END/ERR) in both cases. WiC64 showed a related problem in the opposite direction: the reference example for the WiC64 driver routine (FOTOFIX.C000) consistently expects **lowercase** URLs - join/tick URLs built with uppercase led to a "/join looks fine, /tick consistently fails with ERR UNKNOWN SESSION" bug, because the supposed session ID was actually an error response. For a new platform, it's worth testing letter case early instead of assuming it.

4. Hardware interfaces and event

Each retro computer needs a matching WiFi/network interface to reach the server. Project pages of the interfaces used:

Platform	Interface	Link
Atari XL/XE	FujiNet	fujinet.online
Commodore 64 (standard)	Meatloaf	github.com/idolpx/meatloaf
Commodore 64 (experimental)	WiC64	wic64.de
Schneider/Amstrad CPC	M4 board	cpcwiki.eu/index.php/M4_Board

Built for [Classic Computing 2026](#), an exhibition by Verein zum Erhalt klassischer Computer e.V. Date: 10-11 October 2026, Hall 10 / CD-Kaserne, Hannoversche Strasse 30b, 29221 Celle.

5. Testing without real hardware

For the server, there's a minimal pygame stub that lets it run headless and be driven through end-to-end tests via a raw TCP socket (send HELLO, wait for WAIT/START, etc.), without needing an actual pygame window.

For the BASIC clients, emulator support varies by platform - from practical experience:

Platform	Emulator	Status
Atari (FujiNet)	Fujisan 2.0.5beta	Works reliably on Linux. The same setup does not work on Windows - cause unknown so far.
C64 (WiC64)	VICE	Can emulate the WiC64 client - the experimental client can be tested with it without real hardware.
CPC (M4)	CPCEmu	Simulates an M4 interface, but not tested yet - status open.

For packing a BASIC program into a D64 disk image (e.g. for C64 tests with VICE), d64-inspector (author: P. David Buchan, GPLv3) has proven useful. For the PETSCII view, d64-inspector itself uses the C64 TrueType font family (author: "Style", style64.org) - both external tools, not part of this repo. Ready-made reference disk images (Atari, CPC, the original WiC64 driver) live under disk-images/ in the repo.

Without the real event photo server, PHOTO_SOURCE = "FTP" can be tested locally with a simple FTP server (started from the images directory):

```
# Linux
sudo python -m pyftplib -p 21
# Windows
python -m pyftplib -p 21
```

6. Extending to further retro computers

FujiNet doesn't just exist for the Atari - it's also available for e.g. the Apple II and MS-DOS PCs. The general approach for a new platform:

6.1 Choosing the right method

- **Can the network hardware keep a persistent TCP connection open from BASIC?** Then use method 1 (raw TCP, port 6502) - the protocol is already generic (the platform name is simply sent along) and needs no server change.
- **Can it only make single, one-off HTTP requests?** Then use method 2 (HTTP polling, port 8080) - also usable without a server change, since the bridge is already built platform-neutral.

In both cases the platform name (e.g. "APPLE2" or "DOS") is freely chosen - it's only sent along as text and used for display.

6.2 Needed on the client side

- Read the joystick/input on the target platform (each platform has its own method here - Atari STICK(), C64 a PEEK on a CIA register, CPC JOY()). Convention in this project: always use the computer's first joystick port.
- Send HELLO/MOVE/BYE (method 1) or poll /join+/tick (method 2) - see section 3 for the exact format.
- Optional: a PIN prompt before connecting, for the photo display.

6.3 Optional on the server side (just for looks)

So the new platform gets a matching color and company logo in the window, add two config blocks to the server:

```
PLATFORM_COLORS = {  
    ...  
    "APPLE2": (100, 100, 255), # pick your own color  
}  
LOGO_NAME_CANDIDATES = {  
    ...  
    "APPLE2": ["apple", "apple2"],  
}
```

6.4 Honest expectation

Every platform in this project so far has shown its own, previously undocumented quirks (see section 3.2 on letter case with Meatloaf and WiC64, or e.g. that "PI" is a reserved keyword in Amstrad BASIC and can't be used as a variable name). For a new platform, it's worth building in a visible raw-response diagnostic from the start (just display the raw server response) - in practice, that's proven the fastest way to find real hardware quirks, instead of guessing them.

7. Configuration at a glance

All of the following settings are clearly marked config blocks at the top of `tron_server.py`:

Block	Purpose
<code>BASE_DIR</code>	Base folder (script location) that all relative paths below hang off of
<code>SERVER_BUILD</code>	Version number - increment on every change, shown in console and window
<code>SCROLL_TEXT</code> + <code>SCROLL_*</code>	Demo scroll text (only runs while waiting) and its look
<code>RESULTS_CSV_PATH</code>	Where game results are logged and read back in at startup
<code>FTP_HOST</code> / <code>FTP_*</code>	Login details for the FTP server used for visitor photos
<code>LOGO_DIR</code> / <code>LOGO_NAME_CANDIDATES</code>	Folder and filenames for company logos + club logo
<code>GRID_W</code> / <code>GRID_H</code>	Size of the playfield
<code>TICK_RATE</code>	Movement speed (ticks per second) - affects all platforms equally

State: matches the server build current at the time this document was created. For bigger protocol changes, it's worth regenerating this document.